DAY 05 · BEGINNER · FREE API

Structured Output

Describe the shape you want once, get an object back, and learn the two ways it fails without telling you.

Pydantic | with_structured_output | Literal | include_raw | Notebook

KGP Talkie
LEARN. BUILD. TALK AI

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

The 10-Day Course Map

KGP Talkie
LEARN. BUILD. TALK AI

WEEK 1 TALKING TO A MODEL

01 Your First AI Program
Ask a model, print the answer

02 Roles and Messages
The system message sets the job

03 Prompt Templates
One template, many inputs

YOU WILL HAVE BUILT
a script that answers anything you type into it

WEEK 2 BUILDING SOMETHING

04 Chains: The Pipe
`prompt | model | parser`

YOU ARE HERE

05 Structured Output
An object, not a paragraph

06 Memory
A checkpointer and a thread id

YOU WILL HAVE BUILT
a chatbot that remembers what you said a minute ago

WEEK 3 GIVING IT ABILITIES

07 Tools
Your Python, called by the model

08 Your First Agent
It decides when to call them

09 Chunks and Vectors
Find a paragraph by meaning

10 RAG with RAGWire
Citations, and a refusal

One notebook and one deck a day, **60 to 90 minutes** each. Every model used is a **free** one, and Docker only turns up on Day 9.

Why Paragraphs Are Not Data

WHAT YOU GOT

"This review is moderately positive. Here's the breakdown: Negative, the delay. Positive, the quality, and the customer would buy again..."

Cannot sort it. Cannot count it.

To use it, parse English.

WHAT YOU WANT

```
ReviewScore(
rating=4,
positive=True,
reason='Excellent quality')
```

Store it. Count it. Chart it.

Write an if statement against it.

Four lines of Pydantic get you the right-hand side.

Field Descriptions Are Instructions

YOUR CLASS

```
class ReviewScore(BaseModel):
rating: int = Field(
description="1 is terrible,
5 is excellent")

positive: bool = Field(
description="true if they
would buy again")
```

SENT TO THE MODEL AS JSON SCHEMA

```
"rating": {
"description": "1 is terrible,
5 is excellent",
"type": "integer" }
```

The orange text is **read by the model**.

Vague descriptions produce vague fields.

Not a comment. It is **sent to the model**, and it is where your effort goes.

Constrain Answers with Literal



`category: str`

"billing"

"Billing Department"

"money problem"

"Payments / Finance"

now write the normaliser

`category: Literal["delivery", "billing", ...]`

`"billing"`

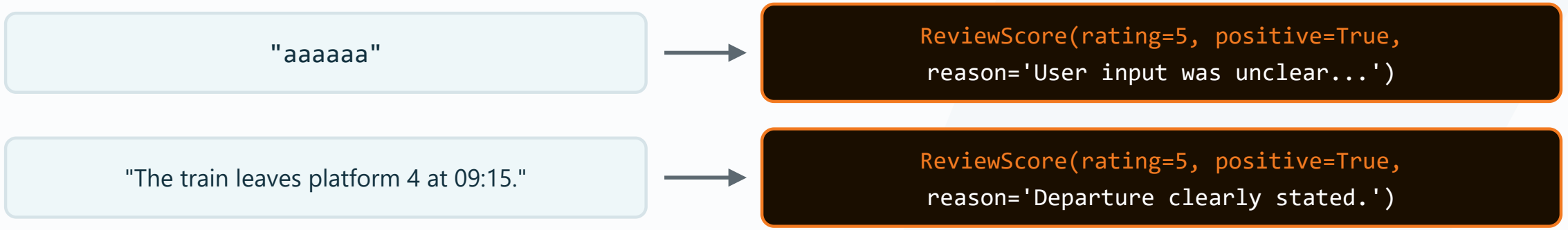
`"delivery" | "product" | "account"`

One of four strings. Always.

a model you can safely route on

A `Literal` gives you something you can write an `if` statement against.

A Valid Object Can Still Be Wrong



Structure guarantees the **shape**, never the **sense**.

The fix is a field whose only job is to let it decline: **`is_a_review: bool`**

Nothing in the pipeline can tell you **"this input was not a review"**.

Tomorrow: Memory

KGP Talkie
LEARN. BUILD. TALK AI

WHAT IT CAN DO NOW

Turn text you give it
into clean, typed data

```
Ticket(category='billing',
       urgency='high')
```

WHAT IT STILL CANNOT

Remember the last message

Look up a live rate

Read a file of yours

Day 6 · Days 7 and 8 · Days 9 and 10

Tomorrow it gets **memory**, and you find out what remembering actually costs.